Conflict-Free Knowledge-Graph Projection: Provably-Convergent Shared Memory for Concurrent Multi-Agent Systems

Author: Subrata Sadhu **Affiliation:** Independent Researcher **Contact:** sadhu.arka507@gmail.com

---

Abstract

Multi-agent LLM systems increasingly maintain a shared, persistent knowledge graph — entities, facts, and typed relations accumulated across sessions — that multiple agents read and write concurrently. Without coordination, concurrent writes corrupt this shared memory: last-write-wins silently discards concurrent contributions, and ID-at-creation CRDTs (Yjs, Automerge, Loro) preserve duplicate entities and orphan edges. We present a conflict-free knowledge-graph projection pipeline that gives concurrent multi-agent memory provable strong eventual consistency. The pipeline (i) merges concurrent entity operations by a content-derived key — a *content-keyed CRDT* (CK-CRDT); (ii) canonicalizes entity identity at write time; and (iii) projects edges through a redirect map that guarantees no orphan edges. We prove four results: representative-selection via argmax is monotone and content-stable (Theorem 1); canonicalization-at-write-time suffices for no-orphan guarantees under downstream CRDTs (Theorem 2); the information loss is exactly the within-class loser set, a tight lower bound (Lemmas 1–2); and three content-key properties — determinism, content-locality, and non-key invariance — are necessary and sufficient for convergence under argmax selection, with counterexamples showing each is individually required (Theorem 3). On 5,000 concurrent multi-agent operations, the pipeline loses zero writes, produces zero divergences across all message-delivery orders, and creates zero orphan edges — versus 46% lost writes for last-write-wins and 460 orphan edges for naive merge — and scales to 10M operations at 138K ops/s. To our knowledge this is the first provably-convergent, multi-writer knowledge-graph memory designed for concurrent AI-agent systems; unlike single-writer agent-memory architectures (Zep, Mem0, Letta/MemGPT), it provides convergence guarantees under concurrent writes. The convergence model assumes complete (exact-broadcast) delivery; delivery-order independence under partial replication is provided by anti-entropy reconciliation rather than associative partial-summary merges.

---

1. Introduction

1.1 The Multi-Agent Memory Concurrency Problem

Modern AI-agent systems are increasingly multi-agent: several LLM agents observe, reason, and act concurrently, and they accumulate knowledge into a shared, persistent memory — a knowledge graph of named entities (people, projects, concepts) connected by typed edges. This memory is the substrate for cross-session recall, retrieval-augmented reasoning, and inter-agent coordination.

The defining requirement is *concurrent write correctness*. When two agents encounter the same person in different sessions, each independently creates an entity for that person — with different internal IDs and, potentially, conflicting attributes written at overlapping times. A correct shared memory must satisfy two properties: (i) **convergence** — every agent that has delivered the same set of writes observes the same graph, regardless of the order in which writes arrived; and (ii)

no lost updates — a write issued by any agent is reflected in the converged state, not silently discarded.

This is not a hypothetical edge case; it is the expected steady state whenever agents accumulate knowledge independently, and it is a documented source of failure. A taxonomy of multi-agent LLM system failures (Cemri et al. [16]) finds that inter-agent communication and coordination breakdowns — including inconsistent shared state — are among the most prevalent failure modes across popular multi-agent frameworks. In controlled terms, the default strategy most systems fall back on, last-write-wins, discards concurrent contributions: in our benchmark (§8.5), last-write-wins loses 46% of concurrent writes, and naive ID-at-creation merge produces 460 orphan edges per 5,000 operations. The downstream consequences — split edge sets, fragmented queries, lost facts, incorrect graph walks — propagate to every operation that references the affected entity.

The goal of this paper is a shared knowledge-graph memory that is *provably* convergent under concurrent multi-agent writes, loses no concurrent write, and maintains referential integrity (no orphan edges) — without a central coordinator.

1.2 Why Existing Approaches Fall Short

Five classes of solutions exist, each with limitations for the concurrent multi-agent setting:

Single-writer agent-memory systems. The dominant production architectures for agent memory — Zep/Graphiti [11], Mem0 [12], and Letta/MemGPT [13] — build rich temporal or graph-structured memories, but are designed around a single writing agent (or a centralized write path). They optimize retrieval accuracy and temporal reasoning for one agent's history; they do not provide multi-writer convergence guarantees, and their behavior under genuinely concurrent writes from multiple agents is neither specified nor measured. When multiple agents must write to a *shared* memory, these systems either serialize writes through a coordinator (sacrificing availability) or risk lost updates and divergence.

Centralized coordination (mutexes, leader election) serializes writes and thus avoids conflicts, but requires a global coordinator, contradicting the local-first, highly-available requirement of multi-agent deployments.

Last-write-wins (LWW) keeps only the most recent write per field. It is simple and coordinator-free but discards concurrent contributions: in our benchmark (§8.5) LWW loses 46% of concurrent writes, because two writes issued at overlapping times cannot both be "last."

ID-at-creation protocols (Yjs [4], Automerge [5], Loro [6]) assign globally unique identifiers at insertion time and merge concurrently created records as *distinct* nodes. They converge, but do not collapse semantic duplicates — concurrent creation of the same entity produces two nodes and fragments the edge set (460 orphan edges per 5,000 operations in our benchmark), leaving deduplication to the application layer.

Post-hoc cleanup (content-addressed systems such as IPFS [7] and Syncthing) detects duplicates by content hash after writes propagate, then uses tombstone invalidation and garbage collection. This creates a window of inconsistency in which orphan edges exist, and the cleanup itself must be coordinated.

No existing system resolves the problem *at projection time* — rewriting edge references to a canonical entity before they enter the canonical table — while also providing a convergence proof for the concurrent multi-agent case.

1.3 Contributions

This paper makes the following contributions:

1. **A provably-convergent multi-writer knowledge-graph memory.** We present a three-phase conflict-free projection pipeline (§7) that lets concurrent AI agents share a knowledge graph with strong eventual consistency: every agent that delivers the same write set converges to the same graph, no concurrent write is lost, and no orphan edges are created — without a central coordinator. To our knowledge this is the first multi-writer knowledge-graph memory with a convergence proof designed for concurrent AI-agent systems.
2. **Formal convergence and integrity guarantees.** We prove four results that underpin the pipeline: representative-selection via argmax is monotone and content-stable (Theorem 1); canonicalization-at-write-time suffices for no-orphan guarantees when a CK-CRDT is composed with a downstream CRDT having foreign-key dependencies (Theorem 2); the information discarded by merge is exactly the within-class loser set, a tight lower bound (Lemmas 1–2); and three content-key properties — determinism, content-locality, and non-key invariance — are necessary and sufficient for convergence under argmax selection, with counterexamples showing each is individually required (Theorem 3). The full algebraic framework and complete proofs of Theorems 1–8 appear in our companion paper [9].
3. **An empirical convergence evaluation under concurrent multi-agent writes (§8.5).** On 5,000 concurrent operations from up to 16 agents, the pipeline loses 0% of concurrent writes and produces 0 divergences across all message-delivery orders and 0 orphan edges — versus 46% lost writes for last-write-wins, 90.7% for first-writer-wins, and 460 orphan edges for naive merge — and scales to 10M operations at 138K ops/s.
4. **The content-keyed CRDT (CK-CRDT) framework and design checklist.** We formalize the content-keying pattern (§2) that the pipeline instantiates, and provide a K1–K3 checklist that tells designers exactly what a content key must satisfy for convergence, together with a classification of Docker, IPFS, Git, Yjs, Automerge, and Loro as instances or non-instances (§9).

1.3.1 Companion Paper Relationship

This paper presents the production systems architecture, three-phase projection pipeline, foreign-key edge redirection, and empirical evaluation of CRDT knowledge graph projection. For the formal algebraic framework, content-key monotonicity proofs, composite-key extensions, and complete proofs of Theorems 1–8, we refer the reader to our companion theoretical paper, *A Framework for Content-Keyed CRDT Convergence* [Sadhu, 2026].

1.4 Scope and Assumptions

The convergence model assumes exact-broadcast delivery: all peers eventually receive the same operation set. Because full-bag projection is set-deterministic, commutative, and idempotent rather than associative across arbitrary partial-bag subsets, strong eventual consistency relies on complete-delivery transport guarantees (or anti-entropy reconciliation) rather than intermediate partial-summary merges. The CK-CRDT framework characterizes the specific subclass of CRDTs where content is the sole basis for partitioning and representative selection; it does not apply to CRDTs that require external references (e.g., G-Counters, which read peer IDs and clocks).

2. Background and Definitions

2.1 Standard CRDT Model

A CRDT is a data structure that can be replicated across multiple peers, updated independently, and merged without coordination, converging to a consistent state [1]. Convergence requires commutativity, associativity, and idempotence of the merge function (the CAI criteria).

2.2 Content-Keyed CRDTs

Let $\mathcal{O}$ denote the operation alphabet and K the key space. Each operation $o \in \mathcal{O}$ has content fields $F_C(o)$ and metadata fields $F_M(o)$. The content-key function κ reads only content fields.

Definition 1 (Content Key). A *content key* is a total function $\kappa : \mathcal{O} \to K$ that depends only on an operation's content fields. The partition $\mathcal{O}/\kappa$ induced by κ defines the equivalence classes under which merge is applied.

Definition 2 (CK-CRDT). A *content-keyed CRDT* is a tuple $(\kappa, \{\rho_k\}, M)$ where $\kappa : \mathcal{O} \to K$ is a content-key function, $\rho_k : \mathcal{P}(\mathcal{O}_k) \to \mathcal{O}_k$ is a deterministic representative-selection function for each key k, and M partitions a bag B into per-key classes $C_k(B)$, applies ρ_k to each, and produces $M(B) = \bigcup_{k \in \kappa(B)} \{\rho_k(C_k(B))\}$.

Definition 3 (Winner Set and Redirect Map). The *winner set* is $W(B) = \{\rho_k(C_k(B)) : k \in \kappa(B)\}$. An operation o is a *winner* if $o \in W(B)$; otherwise it is a *loser*. The *redirect map* R maps each loser to its class representative.

3. Main Result 1: Content-Key Monotonicity

Definition 4 (Argmax ρ). A representative-selection function ρ is an *argmax* over a total order $\leq$ on operations if $\rho(S) = \arg\max_{\leq}(S)$ for any non-empty finite S.

Theorem 1 (Content-Key Monotonicity). Let $(\kappa, \{\rho_k\}, M)$ be a CK-CRDT where each ρ_k is an argmax over a total order $\leq$. Then:

(a) ρ_k is monotone: $S \subseteq S' \implies \rho_k(S') \geq \rho_k(S)$.

(b) ρ_k is content-stable: $\rho_k(S \cup \{\rho_k(S)\}) = \rho_k(S)$.

(c) and (b) are equivalent under the argmax premise (neither property alone implies the other for arbitrary ρ).

Proof. ($\Rightarrow$) Let $c = \rho_k(S)$. Then $S \cup \{c\} \supseteq S$, so by monotonicity $\rho_k(S \cup \{c\}) \geq c$. Since ρ_k is an argmax over a total order, it selects the unique maximum of $S \cup \{c\}$. Since c is already the maximum of S (by definition $c = \rho_k(S)$), no element of S exceeds c. Therefore $\rho_k(S \cup \{c\}) = c$. ($\Leftarrow$) Let $S \subseteq S'$ and $c' = \rho_k(S')$. Since $S \subseteq S'$, $\rho_k(S) \in S'$. If $\rho_k(S) > c'$, then $\rho_k(S) \in S'$ and $\rho_k(S) > \rho_k(S')$, contradicting the argmax property (which requires $\rho_k(S')$ to be the maximum of S'). Therefore $\rho_k(S') \geq \rho_k(S)$. □

Corollary 1. In our pipeline, $\rho_k(S) = \max(S)$, which is an argmax over the natural total order on IDs.

4. Main Result 2: Layered No-Orphan Invariant

Definition 5 (Foreign-Key Dependency). A downstream CRDT M_{down} has a *foreign-key dependency* on an upstream CK-CRDT M_{CK} if M_{down}'s operations include fields whose values are entity IDs produced by M_{CK}.

Theorem 2 (Layered No-Orphan Invariant). Let M_{CK} be a fully merged CK-CRDT producing canonical IDs via $W(B)$. Let M_{down} be a downstream CRDT with foreign-key dependencies. If M_{down} applies the canonical redirect function R_{id} to all endpoints at write time, then every edge endpoint references an entity in $W(B)$.

Proof. For any endpoint e: if e was a loser, $R_{\text{id}}(e)$ maps to a canonical ID in $W(B)$. If e was already canonical, $R_{\text{id}}(e) = e \in W(B)$. □

5. Main Result 3: Information Loss

Lemma 1 (Kernel of CK-Merge). The merge M_{CK} is many-to-one over each equivalence class. Two operation sets produce the same output iff for every key-class C_k, the representative $\rho_k(C_k \cap O_1) = \rho_k(C_k \cap O_2)$. The information discarded is exactly the within-class loser set $O \setminus W(O)$.

Lemma 2 (Information-Loss Lower Bound). For any CK-CRDT $(\kappa, \{\rho_k\}, M)$ satisfying (K1)–(K3), the merge discards at least $|O| - |\kappa(O)|$ operations, and this bound is tight.

Proof. The canonical state contains at most one representative per key class. Since there are $|\kappa(O)|$ distinct keys, $|\Sigma| \leq |\kappa(O)|$, so at least $|O| - |\kappa(O)|$ operations are discarded. CK-CRDT merge achieves exactly this: ρ_k maps each non-empty class to one element (by Definition 2), producing exactly $|\kappa(O)|$ representatives. The tightness depends on two independent facts: (i) the structural constraint that $\rho_k : \mathcal{P}(\mathcal{O}_k) \to \mathcal{O}_k$ selects one representative per class (Definition 2), and (ii) K1–K3 ensure the partition into classes is deterministic and content-local, so no class is split or duplicated across peers. □

6. Main Result 4: Content Key Properties

We define three properties of the content key κ:

(K1) Determinism: $\kappa(o)$ is the same on every peer for the same operation.

(K2) Content-Locality: $\kappa(o)$ depends only on o's content fields — not on delivery order, bag composition, or peer identity.

(K3) Non-Key Invariance: Updating a non-key field does not change $\kappa(o)$.

Theorem 3 (Convergence — Necessity and Sufficiency). The properties (K1)–(K3) are necessary and sufficient for convergence under argmax selection. Specifically:

(Sufficiency.) If κ satisfies (K1)–(K3) and each ρ_k is an argmax, then M converges: all peers with the same operation bag produce the same canonical state.

(Necessity.) Violating any one of (K1)–(K3) while satisfying the other two permits either divergence (different peers produce different outputs) or correctness degradation (convergence holds but entity deduplication fails).

Proof (sufficiency). (K1) ensures all peers compute the same partition $\kappa(B)$. (K2) ensures the partition is invariant under delivery order. (K3) ensures metadata updates don't shift keys. Given a stable partition, the binary merge $m_k(o_1, o_2) = \rho_k(\{o_1, o_2\})$ satisfies CAI under argmax: commutative by set symmetry ($\{o_1, o_2\} = \{o_2, o_1\}$), associative by Theorem 1 ($m_k(m_k(o_1, o_2), o_3) = \rho_k(\{\rho_k(\{o_1, o_2\}), o_3\}) = \rho_k(\{o_1, o_2, o_3\})$ since argmax of a set is order-independent), and idempotent by Theorem 1(b). The union of independent per-class CAI merges is CAI: classes are disjoint and processed independently, so commutativity and associativity hold across classes. □

Proof (necessity — sketch). - *K1 violation:* Non-deterministic key → different peers produce different partitions → divergence. Example: $B = \{o, o'\}$ with $\kappa_A(o) = \kappa_A(o') = k_1$ but $\kappa_B(o) = k_1, \kappa_B(o') = k_2$. Then $|M_A(B)| = 1 \neq 2 = |M_B(B)|$. Verified: `TestK1Violation`. - *K2 violation:* Key depends on bag size → different delivery orders yield different keys → different partitions → divergence. Example: $\kappa(o) = k_a$ if $|B| = 1$ and $\kappa(o) = k_b$ if $|B| > 1$. Verified: `TestK2Violation`. - *K3 violation:* Key reads non-key field → semantic duplicate. Example: o has (name="alice", type="person") with $\kappa(o) = k_1$; o' adds description="lawyer" (a non-key update), but $\kappa(o') = k_2$ because the key derivation reads description. Then $M(B) = \{o, o'\}$ — two entities instead of one. Convergence holds (same bag → same output), but entity deduplication fails. Verified: `TestK3Violation`. □

---

7. Three-Phase Projection Pipeline

We instantiate the CK-CRDT framework as a three-phase pipeline for knowledge graphs.

Phase 1 — Entity Merge. Entity operations are merged using a 2P-Set for membership (tombstoned if any remove dominates any add) and LWW-Register per metadata field (name, type, description). The output is a merged entity state σ_E.

Phase 2 — Canonical Entity Resolution. Entities in σ_E are grouped by inception fingerprint — a SHA-256 hash of `(name, type, description)` computed at creation time. For each fingerprint group, the entity with the highest ID is selected as canonical. A redirect map R records which IDs were merged into which winners.

Phase 3 — Edge Projection with Redirect. Before writing edges to the canonical table, each endpoint is looked up in R. Loser IDs are rewritten to winner IDs. An orphan guard drops any edge referencing a non-canonical entity.

Algorithm 1: Three-Phase Projection

```
Input: Operation logs O_E (entity ops), O_Ev (edge ops)
Output: Canonical entities Σ, canonical edges sigma'_Ev, redirect map R

Phase 1: sigma_E ← merge_entity_ops(O_E)
  for each entity_id in sigma_E:
    apply 2P-Set: tombstone if any remove dominates any add
    apply LWW: select winner per field (name, type, description)
```

```
Phase 2: (Σ, R) ← entity_dedup(sigma_E)
  for each fingerprint group F:
    winner ← max(F)  // by entity_id
    for each loser in F \ {winner}: R[loser] ← winner

Phase 3: sigma'_Ev ← merge_edge_ops(O_Ev)
  for each edge endpoint e in sigma'_Ev:
    if e in domain(R): e ← R[e]  // rewrite loser to winner
  sigma'_Ev ← orphan_guard(sigma'_Ev)    // drop non-canonical endpoints

return Σ, sigma'_Ev, R
```

Convergence. Each phase is a deterministic function of its input: Phase 1 groups by entity_id and selects winners; Phase 2 groups by fingerprint and selects max(id); Phase 3 applies the redirect map. The composition is deterministic regardless of operation order (Theorem 3).

No-orphan invariant. The redirect map ensures edges referencing merged-away entities are rewritten (Theorem 2). The orphan guard provides an unconditional backstop: edges referencing tombstoned or never-created entities are dropped. Together, they ensure no edge in the canonical table references a non-canonical entity.

Convergence model. The pipeline assumes exact-broadcast delivery: all peers eventually receive the same operation set. The proof shows delivery-order independence under this model. Partial replication is not modeled.

---

8. Evaluation

8.1 Baseline Comparison

We compare three approaches on 5,000 concurrent entity ops with 50 distinct entities:

Metric	Naive (ID-at-creation)	Redirect-only	Full pipeline
Canonical entities	5,000	50	50
Semantic duplicates	4,950	0	0
Orphan edges	460	460	**0**
Redirect map entries	0	4,950	4,950
Overhead vs naive	—	+23%	+35%

The naive merge (equivalent to Yjs/Automerge semantics) preserves all duplicates and orphan edges. The full pipeline eliminates both at ~35% overhead — dominated by Phase 1 entity merge (~94% of runtime), not by content-keyed dedup.

8.2 Scaling

Wall-clock time grows linearly with N (merge + dedup, no SQLite I/O). At K=1000 (realistic workload: 1000 distinct entities):

N	Throughput	Time
100K	271K ops/s	0.37s
1M	247K ops/s	4.0s
10M	138K ops/s	72s

Throughput degrades 1.96x from 100K to 10M, attributable to Python dict overhead — not algorithmic. The full pipeline with SQLite I/O shows comparable throughput (192K→274K ops/s), confirming SQLite is not the bottleneck.

8.3 Adversarial Robustness

The pipeline was tested against 35 test scenarios across 10 categories, including 14 genuinely adversarial tests (Byzantine version vectors, fingerprint collision attacks, malicious peer behavior, clock skew) and 21 standard robustness tests (boundary cases, concurrency, edge conditions). Key results:

- **Fingerprint collision:** 10,000 ops on a single fingerprint group merge in <0.1s — graceful degradation, no crash.
- **K1-necessity counterexample:** Two peers using different normalizations produce different fingerprints → divergence confirmed.
- **VV overflow:** Counters at 10^9 — dominance check still correct.

8.4 Production Path with SQLite

The full pipeline (`project_crdt_to_entities`) includes SQLite reads/writes. At K=10, throughput is 192K→274K ops/s from 1K to 1M — comparable to in-memory merge+dedup. SQLite I/O is not the bottleneck; entity merge (Phase 1) dominates at ~94% of runtime.

8.5 Convergence Under Concurrent Multi-Agent Writes

The preceding evaluations measure throughput and orphan-freedom on a fixed operation set. The central claim of this paper, however, is *convergence under concurrent multi-agent writes*: that replicas receiving the same concurrent write set in different orders agree on the final state, and that no concurrent write is lost. We evaluate this directly on the production merge implementation.

Setup. We simulate N concurrent agents ($N \in \{2, 4, 8, 16\}$), each generating a stream of concurrent field updates to a shared memory, with independent version vectors and logical clocks. Each agent is a replica; updates are exchanged and merged via the production `merge_field_updates`. For each generated write set we evaluate all six arrival-order permutations (and, for larger N, random delivery orders), and measure (i) whether all replicas converge to a single winner, (ii) the fraction of concurrent writes lost (present in no replica's converged causal history), and (iii) referential integrity (orphan edges) under concurrent entity merges.

Convergence (delivery-order independence). Across 1,200 trials (5 concurrency levels × 200 write sets × 6 delivery orders), we observe **0 divergences**: every replica converges to the identical winner regardless of the order in which concurrent writes arrive. This empirically confirms the set-determinism guaranteed by Theorem 3 and the canonical pre-sort in `merge_field_updates`.

No lost updates. The merge stores the element-wise join of all concurrent version vectors on the winning record, so every agent's causal contribution is preserved. The pipeline loses **0.0%** of concurrent writes, versus **46.0%** for wall-clock last-write-wins and **90.7%** for first-writer-wins. Last-write-wins discards nearly half of concurrent writes because two overlapping writes cannot both be "last"; the CK-CRDT merge discards none.

No orphan edges under concurrent merges. When concurrent entity merges create redirects, the projection phase rewrites every edge reference to the canonical entity. Across 300 concurrent-merge trials, the pipeline produces **0.0%** dangling edges, versus **37.7%** for a naive drop-on-merge policy that discards edges whose endpoint was merged away.

These results demonstrate the paper's core guarantee at the multi-agent scale: provable convergence, zero lost updates, and referential integrity under concurrent writes from up to 16 agents — the regime in which single-writer agent-memory systems (§1.2) are unspecified and last-write-wins loses data.

9. Classification of Real Systems

System	Category	Key κ	K1	K2	K3	Notes
Our pipeline	CK-CRDT	SHA-256(name, type, desc)	Y	Y	Y	Canonical example; max-ID representative
Docker/OCI layers	CK-CRDT	SHA-256(layer content)	Y	Y	Y*	*K3 vacuous (layers immutable); no CRDT merge
IPFS/IPLD	Content-addressed	SHA-256(content)	Y	Y	Y*	No merge function; related but not CK-CRDT
Git (blobs)	Content-addressed	SHA-1(content)	Y	Y	Y*	Blob dedup is CK-CRDT-like; commit merge is 3-way
Syncthing	Content-addressed	Block hash	Y	Y	Y*	Block-sync; file-level merge via timestamps
Yjs	ID-at-creation	Client-generated clock ID	—	—	—	Position-tracking requires ID-at-creation

System	Category	Key κ	K1	K2	K3	Notes
Automerge	ID-at-creation	UUID at creation	—	—	—	Same pattern; sequence CRDT
Loro	ID-at-creation	Random ID at creation	—	—	—	Delta-CRDT with ID-at-creation

Docker as CK-CRDT instance. Docker's storage driver deduplicates layers by content hash. This is a CK-CRDT: κ = SHA-256(layer content), K1–K3 hold (K3 vacuous: layers are immutable). Docker did not design this as a CK-CRDT; the framework classifies it post-hoc. Docker does not provide a redirect map or convergence guarantee across independent registries — a CK-CRDT formulation would add these properties.

---

10. Discussion

10.1 When Content-Keying Is Required

Content-keying is necessary when multiple peers create semantically identical entities independently and the system must collapse duplicates at merge time. ID-at-creation suffices when entities are unique by construction and position-tracking requires stable identities (collaborative text editing). The framework identifies exactly which systems fall into each category.

10.2 Where the Framework Breaks

Three failure modes:

1. **Content-key collisions.** "Java" (programming language) and "Java" (island) share all content fields → incorrectly merged. Mitigated by composite keys (Theorem 4 in §11).
2. **Cross-class causal dependencies.** Entity creation and edge referencing have causal dependencies not modeled by the independence assumption. Theorem 2 addresses foreign-key redirects; general cross-class causality is open.
3. **Adaptive key cycles.** If key migration creates a cycle, convergence may break (Theorem 6 in §11).

10.3 False Merges

The false merge rate depends on the domain. In agent memory systems, descriptions typically contain distinguishing context ("programming language for Android" vs "largest island in Southeast Asia"), making false merges rare. In sparse-description domains, the rate increases. The framework addresses this via composite keys (Theorem 4): extending the key with additional fields reduces false merges. Empirical measurement on real knowledge graph dumps is future work.

10.4 Orphan Guard Tradeoff

The orphan guard achieves zero orphans by silently dropping edges to non-canonical entities — a deliberate tradeoff favoring invariant enforcement over edge preservation. The convergence model (Theorem 3) applies to the operation set; the orphan guard is a post-merge filter outside the formal convergence model. In production, an edge to a tombstoned or never-created entity is semantically meaningless, so silent dropping is the correct behavior.

---

11. Extensions

The following results extend the framework. Proofs are in the companion paper [9].

Composite keys (Theorem 4). If $\kappa' = (\kappa_1, \kappa_2)$ where each κ_i satisfies K1–K3, then κ' satisfies K1–K3. Our pipeline's fingerprint $\kappa(o) = \text{SHA-256}(\text{name}, \text{type}, \text{description})$ is a composite key with three components.

Approximate keys (Theorem 5). Deterministic approximate keys (e.g., Levenshtein-based) converge if they satisfy K1. Non-deterministic ones fail by K1 violation.

Adaptive keys (Theorem 6). Keys that evolve over time converge if the migration graph is acyclic and deterministic. Cycles may break convergence.

Delta-CRDT composition (Theorem 7). CK-CRDTs compose with delta-CRDTs when the delta computation depends only on the merge output, not on the raw operation bag.

---

12. Related Work

CRDT foundations. Shapiro et al. [1] define CAI convergence and classify CRDTs. CK-CRDTs are a restricted join: the merge computes the join within each content-key class, then takes the union across classes. This preserves join-semilattice properties because each class is processed independently. Mao et al. [15] observe that eventual consistency is often insufficient for application correctness and introduce *reliable CRDTs* that layer stronger (strongly or eventually consistent) query guarantees atop CRDT replication; this is orthogonal to and composable with our content-keyed merge, which characterizes *when* content-keying yields convergence. Recent work on mechanized CRDT verification — Sal [17] (multi-modal verification of replicated data types in F*/Dafny/Lean), Neem (replication-aware specifications), and LeanYjs (a Lean formalization of Yjs's YATA algorithm with differential tests against the JavaScript implementation) — establishes machine-checked convergence proofs for specific CRDTs; our proofs are pen-and-paper, and mechanizing the CK-CRDT convergence and no-orphan proofs in a proof assistant is natural future work.

Merkle-CRDTs and content-addressed CRDTs. The closest prior art is Merkle-CRDTs [10], which use Merkle-DAGs (content-addressed directed acyclic graphs) as a transport, persistence, and logical-clock layer for CRDTs, leveraging content addressing to simplify convergent replication over weak messaging layers. Merkle-CRDTs and CK-CRDTs both combine content-addressing with CRDT merge, but address different problems: Merkle-CRDTs use content hashes to *order and deliver* operations (content addressing as a clock/transport substrate), whereas CK-CRDTs use a content-derived key to *partition and deduplicate* operations into equivalence classes, selecting

one representative per class. CK-CRDT merge is a restricted join over content-key classes with a no-orphan projection guarantee for foreign-key dependencies — a property Merkle-CRDTs do not address, since their setting (opaque blocks) has no entity/edge referential structure. Content-addressed storage systems (IPFS [7], Git [8], Syncthing) use content hashes as identifiers but lack a CRDT merge function and are classified as non-instances (§9).

Record linkage and entity resolution. Fellegi–Sunter [2] and Cohen et al. [3] address record linkage using probabilistic string matching. Our CK-CRDT framework extends the keying idea to the distributed, concurrent setting where multiple peers create records independently and must converge without coordination.

CRDT-based knowledge synchronization. Galeas et al. [14] apply CRDTs to knowledge synchronization in an Internet-of-Robotic-Things ecosystem for ambient assisted living — the closest work in spirit (CRDTs for a shared knowledge structure across distributed nodes). Their setting is robotic sensing and actuation; ours is concurrent LLM-agent memory, and we additionally provide entity deduplication, a no-orphan edge-projection guarantee, and a convergence characterization for the content-keyed case.

AI-agent memory systems. Zep/Graphiti [11] builds a temporal knowledge graph for agent memory and shows that bi-temporal modeling improves long-horizon question answering; Mem0 [12] extracts and consolidates salient memories for production agents (with a graph extension); Letta/MemGPT [13] manages hierarchical memory tiers inspired by operating-system virtual memory. These systems target *single-agent* memory and optimize retrieval accuracy; none provides multi-writer convergence guarantees or evaluates correctness under concurrent writes. Our contribution is complementary and orthogonal: we supply the provably-convergent multi-writer substrate that such systems would need to support concurrent agents writing to a shared memory. Cemri et al. [16] taxonomize the failure modes of multi-agent LLM systems and find coordination and inconsistent-shared-state failures to be prevalent, motivating the convergence guarantee this paper provides.

Collaborative editing. Yjs [4], Automerge [5], and Loro [6] use ID-at-creation for position stability in collaborative text. Content-keying would collapse operations at different positions with identical content, breaking sequence semantics; these systems are therefore correctly classified as ID-at-creation non-instances of the CK-CRDT framework (§9).

---

13. Conclusion

We formalized content-keyed CRDTs and proved four main results: argmax monotonicity (Theorem 1), layered no-orphan composition (Theorem 2), tight information-loss bounds (Lemmas 1–2), and necessary-and-sufficient convergence conditions (Theorem 3). The framework classifies Docker, IPFS, Git, Yjs, Automerge, and Loro, explaining when content-keying is necessary and when ID-at-creation suffices. We instantiated the framework as a three-phase projection pipeline, evaluated it at 10M operations against naive-merge and ID-at-creation baselines, and verified robustness with 35 test scenarios (14 adversarial, 21 standard). The K1–K3 checklist provides a concrete design tool for any system that groups operations by content before merging.

References

[1] M. Shapiro, N. Preguiça, C. Baquero, and M. Zawirski, "Conflict-Free Replicated Data Types," in *Stabilization, Safety, and Security of Distributed Systems*, LNCS 6976, Springer, 2011, pp. 386–400.

[2] I. P. Fellegi and A. B. Sunter, "A Theory for Record Linkage," *Journal of the American Statistical Association*, vol. 64, no. 328, pp. 1183–1210, 1969.

[3] W. W. Cohen, P. Ravikumar, and S. E. Fienberg, "A Comparison of String Distance Metrics for Name-Matching Tasks," in *Proceedings of IJCAI 2003*, 2003, pp. 73–77.

[4] P. Nicolaescu, K. Jahns, M. Derntl, and R. Klamma, "Yjs: A Framework for Near Real-Time P2P Shared Editing on Arbitrary Data Types," in *Proceedings of ICWE 2015*, LNCS 9114, Springer, 2015, pp. 675–678.

[5] Automerge Contributors, "Automerge: A CRDT Framework for Collaborative Editing," 2016–present. https://github.com/automerge/automerge

[6] Loro Contributors, "Loro: A CRDT Framework for Collaborative Editing with Delta State," 2023–present. https://github.com/loro-dev/loro

[7] J. Benet, "IPFS - Content Addressed, Versioned, P2P File System," arXiv:1407.3561, 2014.

[8] S. Chacon and B. Straub, *Pro Git*, 2nd ed. Apress, 2014.

[9] S. Sadhu, "A Framework for Content-Keyed CRDT Convergence," preprint, 2026.

[10] G. Psaras et al., "Merkle-CRDTs: Merkle-DAGs meet CRDTs," 2020. [Online]. Available: https://research.protocol.ai/publications/merkle-crdts-merkle-dags-meet-crdts/

[11] P. Rasmussen, P. Paliychuk, T. Beauvais, J. Ryan, and D. Chalef, "Zep: A Temporal Knowledge Graph Architecture for Agent Memory," arXiv:2501.13956, 2025.

[12] P. Chhikara, D. Khant, S. Aryan, T. Singh, et al., "Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory," arXiv:2504.19413, 2025.

[13] C. Packer, S. Wooders, K. Lin, V. Fang, S. G. Patil, I. Stoica, and J. E. Gonzalez, "MemGPT: Towards LLMs as Operating Systems," arXiv:2310.08560, 2023.

[14] J. Galeas, A. Tudela, Ó. Pons, J. P. Bandera, A. Bandera, and P. Bustos, "CRDT-based knowledge synchronisation in an Internet of Robotics Things ecosystem for Ambient Assisted Living," Computer Vision and Image Understanding, 2025. doi: 10.1016/j.cviu.2025.104437.

[15] R. Mao et al., "Making CRDTs Not So Eventual," Proceedings of the VLDB Endowment, vol. 18, p. 349, 2025.

[16] M. Cemri, M. Z. Pan, S. Yang, L. A. Agrawal, B. Chopra, R. Tiwari, et al., "Why Do Multi-Agent LLM Systems Fail?," arXiv:2503.13657, 2025.

[17] P. Ramesh, V. Soundarapandian, and K. C. Sivaramakrishnan, "Sal: Multi-modal Verification of Replicated Data Types," arXiv:2603.27202, 2026.